

// devlog · entry 02

Ports, DNS & my first endpoint

how a URL actually finds my laptop – and the 6 lines of code that turned into "Hello World"

PICKS UP FROM

Entry 01 – Foundations

TOOLS

IntelliJ · Spring Initializr

RESULT

localhost:8080/hello

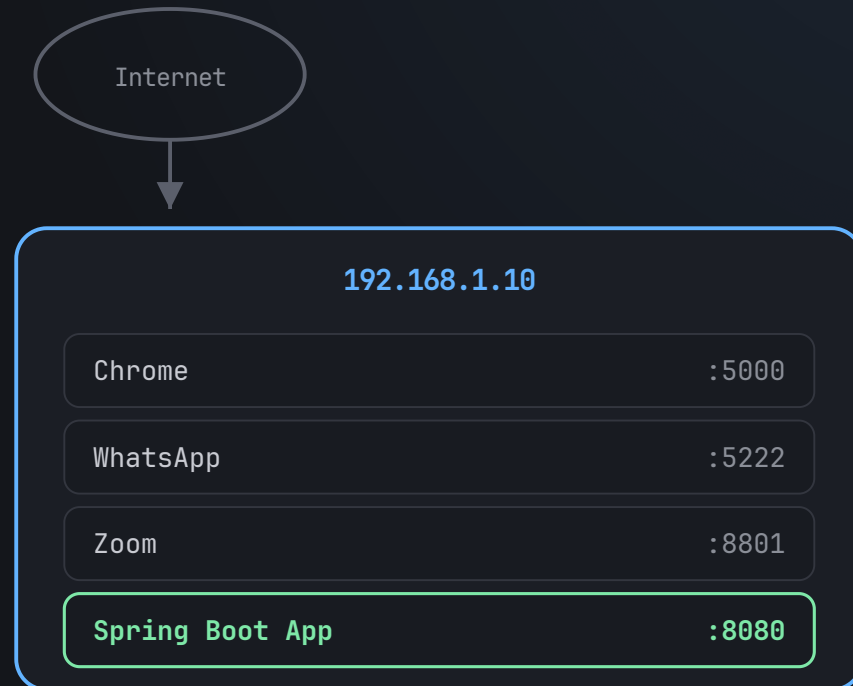
before running anything...

IP finds the machine. Port finds the app.

My laptop has **one IP address**, but Chrome, WhatsApp, Zoom, and my Spring Boot app are all running on it at once. If data arrives, how does the OS know who it's for?

IP address → which machine

Port number → which application on that machine



the most confusing three words

localhost, IP, and port — untangled

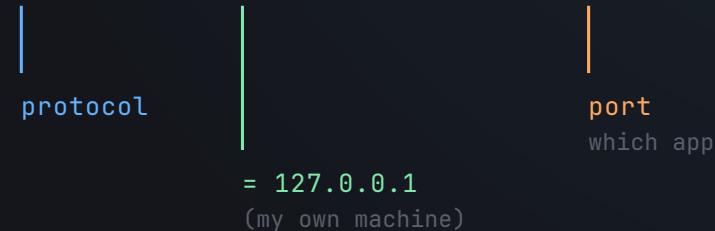
`localhost` just means “my own machine” — it’s shorthand for `127.0.0.1`. No port in a URL doesn’t mean “no port” — the protocol picks a default.

`http://` → port 80

`https://` → port 443

local spring boot → port 8080 (manual)

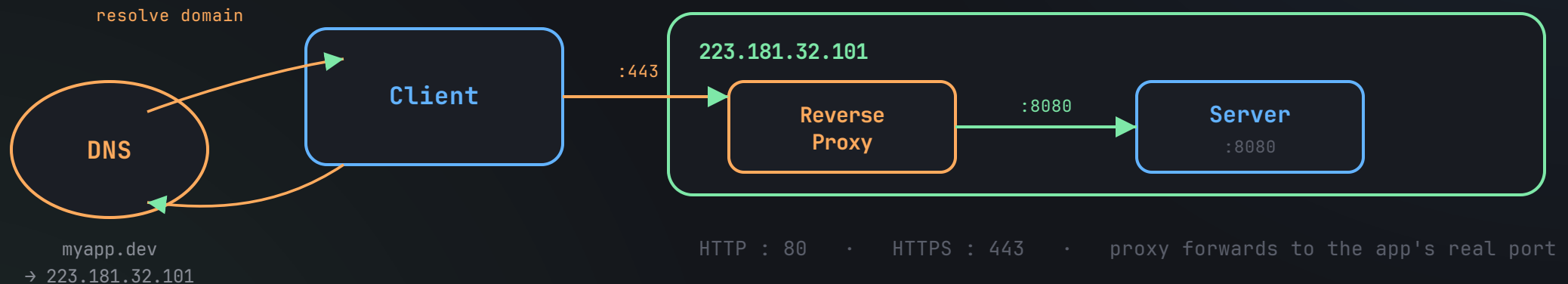
`http://localhost:8080`



“send this request to my own machine, on port 8080.”

what actually happens for a real domain

myapp.dev → DNS → reverse proxy → my app



the public URL never shows :8080 — the reverse proxy quietly maps 443 → 8080 behind the scenes.

before writing one line of logic

Spring Initializr builds the skeleton

Folder structure, Maven config, package layout, metadata — every project needs the same setup. **Spring Initializr** generates it in seconds so I can start on real code.

Project type

Language

Boot version

Java version

Packaging

Metadata

dependency = external library my project needs

Spring Web

Spring Data JPA

Spring Security

MySQL Connector

Lombok

Hibernate

added inside pom.xml

Choosing a version:

SNAPSHOT → work-in-progress, may have bugs

RC → release candidate, almost final

Stable → what I actually pick ✓

opened in intellij for the first time

Four things worth knowing

A generated Spring Boot project looks like a lot of files. Really, there are just a few that matter early on.

src/main/java

my Java source code

src/main/resources

config files + static resources

pom.xml

Maven config – project info + dependencies

application.properties

app config – e.g. change the server port

+ one Main class with main() – annotated @SpringBootApplication

a controller = the receptionist

Reading my first endpoint, line by line

```
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World";
    }
}
```

`@RestController` – this class can handle HTTP requests

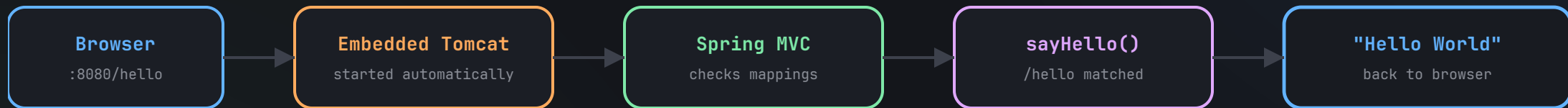
`@GetMapping("/hello")` – run this method on GET /hello

`sayHello()` – the Java method that executes

`return "Hello World"` – the response body sent back

behind localhost:8080/hello

The complete flow of my first request



I never installed Tomcat, wrote sockets, or built a server by hand

Spring Boot did all of it the moment I hit "Run".

it worked in 6 lines... so why go deeper?

Writing @RestController is the easy part

AI tools and IDEs can generate that annotation instantly.
What they can't do for me: explain **why** it broke.

Spring Boot gives speed. Spring fundamentals give control.

404 Not Found – and now I need to ask:

- did Spring detect my controller?
- is the package structure correct?
- did component scanning happen?
- was the bean created?
- did Spring MVC register the mapping?
- did Tomcat even receive the request?

entry 02, wrapped

From an empty folder to a live endpoint

- ✓ IP finds the machine, port finds the app
- ✓ localhost = 127.0.0.1 = my own machine
- ✓ DNS resolves a domain → reverse proxy routes it
- ✓ Spring Initializr generates the whole skeleton
- ✓ @RestController + @GetMapping = one endpoint
- ✓ Embedded Tomcat starts automatically, no setup
- ✓ Spring Boot gives speed – internals give control

next up: Maven · Servlets · Tomcat · IoC · Dependency Injection · Beans – the internals behind today's magic.